



Typo Bugs

Can typing mistakes (typos) cause bugs? Yes, they can! We'll look at some common C programming mistakes in this column.

What does the following program print?

```
int val = 2;
switch(val) {
case 1:
    printf("case 1");
break;
default:
    printf("in default");
}
```

Here, you would expect "in default" to get printed; but it does not print anything and the default case does not execute. Why? The keyword "default" was mistyped as "defalut". Why does the compiler still compile the code without complaining? In C (as in many other languages), a label (as a target of the goto statement) is a name followed by a : (colon). In this case, "defalut:" was treated as a label and hence the bug!

In the May 2009 JoP column, we discussed a typographical (typing) mistake that resulted in the loss of the Mariner-1 rocket because of mistyping a . (full stop) instead of a , (comma). So, we'll go further and discuss some common typing mistakes in C that we as programmers make while writing code.

- 1) Typing 'T' (lowercase 'l') instead of 1 as in the statement:

```
long val = 9l;
```

In this case, the constant is 9 (nine) and not 91 (ninety-one) since 'l' is a suffix to indicate a long variable.

- 2) Typing = instead of ==, which makes a comparison expression into an assignment statement, as in:

```
if (a = 0) // the programmer wanted to check if a == 0
```

This mistake is so very common in C that a "defensive programming" technique is to reverse the condition, as in, `if(0 == a)`, so that compiler will catch it if we make a mistake.

- 3) Typing == instead of =, which makes an assignment statement into a comparison statement, as in:

```
x == 2;
```

In fact, this example is from Peter van der Linden's book *Deep C Secrets* in which he talks about a bug that was holding up a \$20 million sales deal at Sun. It turned out to be this mistake in the code in the I/O library.

- 4) Typing a ; (semi-colon) after a for loop by mistake, as in:

```
for(i = 0; i < 10; i++);
printf("hello");
```

And this code prints "hello" once and not ten times.

- 5) Forgetting to type a comma between two strings, which leads to "stringization" of two strings, as in:

```
char *str[] = {"rain","deer"};
int i = 0;
while(str[i])
    puts(str[i++]);
```

This prints "raindeer" instead of "rain" and "deer".

- 6) Can you find out what's wrong with this code?

```
struct Point {
    int x, y;
}
foo() {
```

Here, we forgot to put a semicolon after the struct definition point. In old C, if we don't provide a return type, it is considered to return int by default. However, the definition for function foo is that it returns the struct Point! (Yes, we can define a struct/class "on-the-fly" as a return type in C/C++). This typo bug has caused many sleepless debugging nights for programmers worldwide.

- 7) As of this writing, the latest news is about "Tiny typo blamed for massive IE security failure" [1]. The following code and the explanation is from [2]:

```
hr = pStream->Read((void*)&pbArray, ULONG.cbSize, NULL);
```

I'll give you one more clue - it's a one-character typo...The first argument is incorrect. It should be:

```
hr = pStream->Read((void*)&pbArray, (ULONG)cbSize, NULL);
```

The extra '&' character in the vulnerable code causes the code to write potentially untrusted data, of size cbSize, to the address of the pointer to the array, pbArray, rather than write the data into the array, and the pointer is on the stack. This is a stack-based buffer overrun vulnerability.

- [1] http://www.theregister.co.uk/2009/07/30/typo_caused_massive_ms_bug
- [2] <http://blogs.msdn.com/sdl/archive/2009/07/28/atl-ms09-035-and-the-sdl.aspx> 

About the author:

S G Ganesh is a research engineer in Siemens (Corporate Technology). His latest book is "60 Tips on Object Oriented Programming", published by Tata McGraw-Hill. You can reach him at sganesh@gmail.com.